PostgreSQL. What if there was a built-in way to rebuild indexes without disrupting any queries that are using the Indexes?

Fortunately from PostgreSQL 13 onward, you can use the CONCURRENTLY keyword with the REINDEX command to accomplish this, rebuilding Indexes online without requiring a third party tool like pg_repack.

Try putting this into action. From psql, run the following command to reindex the index_trips_on_driver_id index. The Verbose is used below and it's optional but shows more information about what's happening.

```
REINDEX (VERBOSE) INDEX CONCURRENTLY index_trips_on_driver_id;
```

Instead of Reindex with an Index, try specifying a Table to reindex all the indexes for the table. Run this statement from psql:

```
REINDEX (VERBOSE) TABLE CONCURRENTLY trips;
```

Running Manual Vacuums

PostgreSQL lays out data on disk in pages that are 8kb in size. The pages are not completely filled by default; space is left available to store new tuples from row modifications. Dead tuples take up space in those pages as well. When Autovacuum runs a VACUUM worker for each table in PostgreSQL, one of the jobs it performs is to reclaim the space used by dead tuples. Once completed, the space that was used by those dead tuples is marked as available again for new row modifications.

As a refresher, performing a manual VACUUM for a table like users is done by running this command from psql:

sql/vacuum_table.sql

```
VACUUM users;
```

The ANALYZE option can be added, and this updates table statistics for the table. The statistics include estimates that are used by the query planner as you saw earlier.

sql/vacuum_analyze_table.sql

```
VACUUM ANALYZE users;
```

New versions of PostgreSQL continue to add capabilities to Vacuum and Analyze. Version 11 made it possible to specify multiple tables at once. Version 12 added SKIP_LOCKED support which skips locked tables (See: *Postgres 12*